

Python 30 Days - Master Notes

DAY 1: Python Fundamentals + Conditional Statements + Git & GitHub

Color guide: Explanation | [Code](#) | [Output](#) | [Important](#) / [Revision](#)

How to use these notes

Read the explanation first, then study the code, predict the output, and finally run the same example in VS Code. The code and output boxes are centered on the page for easier reading and a cleaner revision layout.

1. print() - Displaying Output

print() is used to display text, numbers, or calculation results on the screen. Text is normally written inside quotes. Numbers do not need quotes.

Code

```
print("Hello World")
print("I am learning Python")
print(100)
print(10 + 20)
```

Expected output

```
Hello World
I am learning Python
100
30
```

Important to remember

Quotes make something text. print(10 + 20) performs the calculation first and prints 30.

2. Comments

Comments are notes for humans reading the program. Python ignores comments when the program runs. A single-line comment starts with #.

Code

```
# This is a comment
print("Python") # This is an inline comment
# print("This line will not run")
```

Expected output

```
Python
```

Important to remember

Use comments to explain why code exists. Do not fill every line with unnecessary comments.

3. Variables

A variable is a name used to store a value. Think of it as a labeled box. The value can be changed later by assigning a new value.

Code

```
name = "Bhanu"  
age = 23  
salary = 30000  
print(name)  
print(age)  
age = 24  
print(age)
```

Expected output

```
Bhanu  
23  
24
```

Important to remember

Use = to assign a value. Variable names are case-sensitive: age and Age are different.

4. Basic Data Types

A data type tells Python what kind of value it is working with. The main Day 1 types are str, int, float, bool, and None.

Code

```
name = "Bhanu"      # str - text  
age = 23             # int - whole number  
price = 99.50        # float - decimal number  
is_learning = True   # bool - True/False  
value = None         # no value  
  
print(type(name))  
print(type(age))
```

Expected output

```
<class 'str'>  
<class 'int'>
```

Important to remember

"10" + "20" gives "1020" because they are strings. 10 + 20 gives 30 because they are integers.

5. User Input

input() pauses the program and lets the user type a value. The important rule is that input() returns a string, even when the user types digits.

Code

```
name = input("Enter your name: ")  
print(f"Hello {name}")
```

Expected output

```
Enter your name: Bhanu  
Hello Bhanu
```

Important to remember

If you need to perform number calculations, convert the input using int() or float().

6. Type Conversion

Type conversion changes a value from one data type to another. You practiced `int()`, `float()`, and `str()`.

Code

```
age = "23"
age = int(age)
print(age, type(age))

price = "99.50"
price = float(price)
print(price, type(price))

number = 100
number = str(number)
print(number, type(number))
```

Expected output

```
23 <class 'int'>
99.5 <class 'float'>
100 <class 'str'>
```

Important to remember

Use `int(input(...))` when you expect a whole-number input and want to compare or calculate with it.

7. Arithmetic and Assignment Operators

Operators perform operations on values. Arithmetic operators calculate numbers. Assignment operators store or update values.

Code

```
a = 10
b = 3

print(a + b) # addition
print(a - b) # subtraction
print(a * b) # multiplication
print(a / b) # division
print(a // b) # floor division
print(a % b) # remainder
print(a ** b) # power

a += 5
print(a)
```

Expected output

```
13
7
30
3.3333333333333335
3
1
1000
15
```

Important to remember

`/` gives normal division, `//` gives floor division, `%` gives the remainder, and `**` means power.

8. Strings

A string is a sequence of characters written inside quotes. You practiced indexing, slicing, length, string methods, replacement, concatenation ideas, and f-strings.

Code

```
name = "Bhanu Prakash"
print(name[0])
print(name[-1])
print(name[0:5])
print(len(name))
print(name.upper())
print(name.lower())
print(name.title())

message = "I am learning Java"
print(message.replace("Java", "Python"))

age = 23
print(f"My name is {name} and I am {age} years old.")
```

Expected output

```
B
h
Bhanu
13
BHANU PRAKASH
bhanu prakash
Bhanu Prakash
I am learning Python
My name is Bhanu Prakash and I am 23 years old.
```

Important to remember

Indexing starts at 0. -1 means the last character. In [start:stop], the stop position is excluded.

9. Comparison Operators

Comparison operators compare two values. Their result is always True or False.

Code

```
age = 23
print(age == 23)
print(age != 18)
print(age > 18)
print(age < 30)
print(age >= 18)
print(age <= 30)
```

Expected output

```
True
True
True
True
True
True
```

Important to remember

= means assignment. == means comparison. This difference is very important.

10. Conditional Statements - if / elif / else

Conditional statements allow Python to make decisions. if checks a condition. elif checks another condition when an earlier one was false. else runs when the previous conditions were false.

Code

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible")
else:
    print("You are not eligible")

marks = int(input("Enter your marks: "))

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Expected output

```
Example 1:
Enter your age: 23
You are eligible

Example 2:
Enter your marks: 80
Grade B
```

Important to remember

Python checks an if/elif chain from top to bottom and stops after the first True branch.
Indentation is required.

11. Logical Operator - and

and combines conditions. The complete condition is True only when both sides are True.

Code

```
age = int(input("Enter your age: "))
python_score = int(input("Enter your Python score: "))

if age >= 18 and python_score >= 70:
    print("You are eligible")
else:
    print("You are not eligible")
```

Expected output

```
Age 23, score 88:
True and True -> True
You are eligible

Age 18, score 33:
True and False -> False
You are not eligible
```

Important to remember

and = BOTH conditions must be True.

12. Logical Operator - or

or combines conditions. The complete condition is True when at least one side is True.

Code

```
has_python = True
has_java = False

if has_python or has_java:
    print("You know at least one programming language")
else:
    print("You need to learn a programming language")
```

Expected output

```
You know at least one programming language
```

Important to remember

True or False is True. Only False or False is False.

13. Logical Operator - not

not reverses a Boolean value. not True becomes False, and not False becomes True.

Code

```
is_blocked = False

if not is_blocked:
    print("User can access the website")
else:
    print("User is blocked")
```

Expected output

```
User can access the website
```

Important to remember

Because is_blocked is False, not is_blocked becomes True.

14. Day 1 Final Combined Program - UPDATED

This is the exact final program you tested. It combines variables, input, type conversion, comparisons, logical and, conditional statements, f-strings, print(), and comments.

Code

```
# =====
# DAY 1 - FINAL COMBINED PYTHON PRACTICE
# Candidate Interview Checker
# =====

# Get information from the user
name = input("Enter your name: ")
age = int(input("Enter your age: "))
python_score = int(input("Enter your Python score: "))
city = input("Enter your city: ")

# Check interview eligibility
# Both conditions must be True:
# 1. Age must be 18 or above
# 2. Python score must be 70 or above

if age >= 18 and python_score >= 70:
    print(f"Hello {name}")
    print(f"City: {city}")
    print("You are eligible for the interview.")
else:
    print(f"Hello {name}")
    print(f"City: {city}")
    print("You are not eligible for the interview.")
```

Expected output

```
Enter your name: Bhanu Prakash
Enter your age: 23
Enter your Python score: 70
Enter your city: Bhangalore
Hello Bhanu Prakash
City: Bhangalore
You are eligible for the interview.
```

Important to remember

23 >= 18 is True and 70 >= 70 is also True. Because both sides of and are True, the eligible branch runs. The city is printed exactly as typed, so "Bhangalore" is input spelling, not a Python error.

15. Git & GitHub - Day 1

Git tracks your project history locally. GitHub stores the repository online. You completed the initial repository setup and successfully pushed Day 1 code to the main branch.

Commands practiced

```
git --version
git config --global user.name
git config --global user.email
git init
git status
git add .
git commit -m "Day 1: Python fundamentals and conditional statements"
git remote add origin <repository-url>
git remote -v
git branch -M main
git push -u origin main
```

Daily GitHub routine from now on

```
git status
git add .
git status
git commit -m "Day X: description of work"
git push
```

Important to remember

You do not run git init, git remote add origin, or git branch -M main every day. Those were one-time setup steps. At the end of each day, push that day's code, updated README, and updated notes/PDF.

Day 1 - Common Mistakes

Mistake	Why it is wrong / fix
Using = instead of ==	= assigns a value; == compares two values.
Forgetting int() for numeric input	input() returns a string. Convert it before numeric comparison/calculation.
Wrong indentation	Code inside if/elif/else must be indented consistently.
Running the wrong file	Check the exact filename in the python command before testing.
Thinking and means either condition	and requires both conditions to be True.
Thinking or requires both	or requires at least one condition to be True.
Running git init every day	git init is a one-time repository setup command.

Day 1 - Quick Revision

Item	Meaning
print()	Display output
#	Comment
variable	Stores a value
str / int / float / bool / None	Basic data types
input()	Take user input; returns a string
int() / float() / str()	Type conversion
type()	Check data type
len()	Find length
= / ==	Assignment / comparison
if / elif / else	Decision making
and	Both conditions must be True
or	At least one condition must be True
not	Reverse a Boolean
git add	Stage changes
git commit	Save a local checkpoint
git push	Upload committed changes to GitHub

Self-check questions

1. What is the difference between = and ==?
2. What data type does input() return?
3. What does and require?

4. What does or require?
5. What does not do?
6. Why do we use `int(input(...))`?
7. What happens when an if condition is False and an else block exists?
8. Why is indentation important in Python?
9. What is the difference between `git add`, `git commit`, and `git push`?
10. Which Git setup commands do you NOT need to repeat every day?

Day 1 learning status: Python concepts, conditional practice, final combined program, Git/GitHub setup, first code push, and revision notes are complete. Add this PDF to the notes folder and push it to GitHub to complete the Day 1 repository record.